

category: avalonia

tags: [keyboard, remote-control, x11, xtest, input]

severity: high

created: 2026-07-24

updated: 2026-07-24

project-origin: 6.RemoteControlTool (IPGS.RemoteControl.CcuUI)

Avalonia KeyEventArgs.KeySymbol trả ASCII control-char khi giữ Ctrl — không phải chữ cái gốc

Tình huống gặp phải

Đang fix bug remote-control tool (CCU điều khiển ZCU qua X11/XTest): user báo Ctrl+C, Ctrl+V (và các shortcut Ctrl+chữ khác) không hoạt động khi gõ trên cửa sổ Remote Control để điều khiển máy ZCU từ xa.

`RemoteScreenControl` (Avalonia UserControl) bắt `KeyDownEvent/KeyUpEvent`, map qua `KeyboardMapper.Resolve(Key key, string? keySymbol)` để ra X11 keysym rồi gửi qua giao thức `KEY_EVENT` cho `KeyboardInjector` (ZcuAgent) bơm bằng `XTestFakeKeyEvent`.

Triệu chứng / Lỗi

Giữ Ctrl rồi bấm C/V trên cửa sổ Remote Control: KHÔNG thấy hành động copy/paste xảy ra trên màn hình ZCU. Không có exception, build vẫn PASS. Log phía ZcuAgent (nếu bật) có khả năng thấy warning "`keysym 0x.. không map được keycode trên keymap hiện tại — bỏ qua`" cho phím chữ đang giữ Ctrl.

Nguyên nhân gốc rễ (Root Cause)

`KeyboardMapper.Resolve` có 3 tier ưu tiên:

1. `SpecialKeyMap` (Tab, Ctrl, Alt, F1-F12, ...)
2. `KeyEventArgs.KeySymbol` (ký tự Unicode 1 ký tự) → dùng trực tiếp làm keysym Latin-1
3. Fallback map `Key` enum → chữ cái ASCII thường

Vấn đề: trên Windows, khi giữ Ctrl và bấm 1 phím chữ, hệ điều hành/Avalonia KHÔNG trả về ký tự chữ cái gốc trong `KeySymbol` mà trả **mã ký tự điều khiển ASCII** (Ctrl+A=0x01 ... Ctrl+Z=0x1A, ví dụ Ctrl+V=0x16 "SYN", Ctrl+C=0x03 "ETX"). Tier 2 của `Resolve` chỉ kiểm tra `keySymbol is { Length: 1 }` mà KHÔNG kiểm tra modifier đang giữ, nên nó dùng thẳng mã control-char (0x16) làm keysym gửi đi — SAI, vì keysym X11 đúng cho phím V là 0x76 ('v'). ZcuAgent gọi `XKeysymToKeycode` cho keysym control-char 0x16 → hầu hết layout không có physical key ánh xạ tới đó → trả về 0 → bị bỏ qua (skip), Ctrl+V không hề được gửi tới ZCU.

Trớ trêu là code đã có sẵn Tier 3 (fallback theo `Key` enum → chữ thường ASCII) với đúng docstring giải thích ý đồ thiết kế: *"X server + XTest handle shift-state separately via explicit Shift_L/Shift_R press events sent before this key"* — nghĩa là thiết kế ban đầu ĐÃ dự tính dùng Tier 3 cho trường hợp có modifier, nhưng Tier 2 lại vô tình "che" Tier 3 vì `keySymbol` không null (chỉ sai giá trị).

Giải pháp

Thêm tham số `KeyModifiers modifiers` vào `Resolve()`. Khi `Control` đang giữ, BỎ QUA Tier 2 (`KeySymbol`), nhảy thẳng xuống Tier 3 (map theo `Key` enum → chữ ASCII thường), để `keySYM` gửi đi luôn đúng chữ cái vật lý bất kể Ctrl có giữ hay không.

```
public static uint? Resolve(Key key, string? keySymbol, KeyModifiers modifiers = KeyModifiers.None)
{
    if (SpecialKeyMap.TryGetValue(key, out uint special))
        return special;

    bool controlHeld = (modifiers & KeyModifiers.Control) != 0;

    // Tier 2 SKIP khi Ctrl giữ - KeySymbol lúc này là control-char, không phải chữ cái
    if (!controlHeld && keySymbol is { Length: 1 })
    { /* ... như cũ ... */ }

    return FallbackFromKey(key); // Tier 3
}
```

Truyền `e.KeyModifiers` từ `KeyEventArgs` xuyên suốt: `OnKeyDown/OnKeyUp` (code-behind) → `RemoteScreenViewModel.HandleKeyDown/HandleKeyUp` → `KeyboardMapper.Resolve`.

Áp dụng lại (How to reuse)

- Khi thấy code map `KeyEventArgs.KeySymbol` (hoặc bất kỳ API tương tự trả "ký tự đã áp dụng modifier") sang `keySYM/keycode` để FORWARD phím đi nơi khác (remote control, VNC-like, input injection) → luôn kiểm tra modifier Ctrl/Alt đang giữ TRƯỚC khi tin vào `KeySymbol`.
- Chỉ tin `KeySymbol` khi modifier là None hoặc chỉ Shift/AltGr (những trường hợp OS thực sự trả ký tự đã shift đúng, VD Shift+2 → "@"). Ctrl (và Ctrl+Alt = AltGr trên 1 số layout cần xét riêng) làm `KeySymbol` không còn là ký tự in được.

Chú ý / Cạm bẫy (Gotchas)

- ⚠ Bug này KHÔNG lộ ra qua build (0 error) và KHÔNG throw exception — chỉ lộ khi test tay bằng bàn phím thật với tổ hợp Ctrl+chữ.
- ⚠ Đừng nhầm với dedupe theo `_downKeysyms`/OS auto-repeat — đó là cơ chế đúng, không liên quan tới bug này.
- ⚠ Bug thứ 2 phát hiện cùng lúc (không liên quan): pointer events (`PointerPressed/Released/Moved`) trong `RemoteScreenControl` không set `e.Handled = true` → sự kiện bubble tiếp lên Window/parent, khiến máy điều khiển (CCU) cục bộ CŨNG xử lý click y như UI thường ("cả 2 bên cùng nhận click"). Luôn set `e.Handled = true` cho pointer/key event khi UserControl đã "chiếm" toàn quyền xử lý input để forward đi nơi khác — nếu không, mọi ancestor (kể cả Window) vẫn nhận và có thể phản ứng cục bộ ngoài ý muốn.

Tham chiếu

- File: `IPGS.RemoteControl.CcuUI/KeyboardMapper.cs`, `Views/RemoteScreenControl.axaml.cs`, `ViewModels/RemoteScreenViewModel.cs`
- Project liên quan: 6.RemoteControlTool (TDD §17.3, §17.5)